

Arm[®] GPU Bug Bounty FAQ

Public

Copyright © 2025 Arm Limited (or its affiliates). All rights reserved.

Document Version: 20250623-1.0

- [FAQ on Using the Arm[®] Mali[™] SW Drivers](#)
 - [Where can I find the source releases for Kbase?](#)
 - [How can you determine the version of Kbase?](#)
 - [What is the "dummy model"?](#)
 - [Where is the "dummy model" code?](#)
 - [How do I keep the GPU powered/power it off when idle?](#)
 - [How do I access all IOCTLs/get around the -ENOTTY error?](#)
 - [What is 'SAME_VA' / 'BASE_MEM_SAME_VA' memory?](#)
 - [How do I allocate GPU memory on 64-bit userspace/'SAME_VA' memory?](#)
 - [How do I grow memory on GPU Page Fault?](#)
 - [How do I change the timeout on FENCE_SIGNAL KCPU commands?](#)
- [FAQ on Mali GPUs](#)
 - [What is the Bifrost GPU Architecture?](#)
 - [What is the Valhall GPU Architecture?](#)
 - [What is the 5th Gen Arm GPU Architecture?](#)
 - [What is a "Job Manager" GPU?](#)
 - [What is a "CSF" GPU?](#)
 - [What is "TrustZone[®] Media Protection" \(TZMP\)?](#)
 - [What is "Ace-Lite" ?](#)
 - [What is Tile-Based rendering?](#)
 - [What is Deferred Vertex Shading \(DVS\)?](#)
- [Proprietary notice](#)

This document is Non-Confidential.

Copyright © 2025 Arm Limited or its affiliates. All rights reserved.

This document is protected by copyright and other intellectual property rights. Arm only permits use of this document if you have reviewed and accepted [Arm's Proprietary notice](#) found at the end of this document.

These materials and information are provided as part of Arm's Bug Bounty program to assist security researchers in their vulnerability research related to this program. These documents do not constitute official Arm documentation, and no warranties or guarantees are provided regarding their accuracy, completeness, or suitability for any purpose outside of the Bug Bounty context. Arm reserves the right to modify or withdraw any materials at its discretion without prior notice.

FAQ on Using the Arm[®] Mali[™] SW Drivers

Where can I find the source releases for Kbase?

The latest source releases for Kbase, and all past releases can be found at the following:

- For Bifrost GPUs: <https://developer.arm.com/downloads/-/mali-drivers/bifrost-kernel>
- For Valhall GPUs: <https://developer.arm.com/downloads/-/mali-drivers/valhall-kernel>
- For Mali 5th Gen GPUs: <https://developer.arm.com/downloads/-/mali-drivers/5th-gen-gpu-architecture-kernel>

Refer to [FAQ on Mali GPUs](#) for more information on Arm's GPU Architectures.

How can you determine the version of Kbase?

On a running system, the version may be found from:

```
cat /sys/module/mali_kbase/version
```

And should output something like:

```
r52p0-00eac0 (UK version 1.33)
```

From the source, the version is stored in `drivers/gpu/arm/midgard/Kbuild` as `MALI_RELEASE_NAME`, for example:

```
# Driver version string which is returned to userspace via an ioctl
MALI_RELEASE_NAME ?= "r52p0-00eac0"
```

What is the "dummy model"?

This is the "No Mali" configuration of the driver.

This can be used to test the driver on a system where no GPU is present, such as on a Virtual Device.

The limitation is that it cannot be used to test the system where a vulnerability involves the GPU or GPU Firmware making reads, writes, or executing instructions.

In this configuration, GPU IRQ handlers and register accesses to the real GPU are replaced with "dummy" accesses to a simple model of the GPU.

Each operation in the "dummy model" completes instantaneously and does not emulate any effects on the rest of the system.

The GPU Firmware itself is not executed in the "dummy model".

Where is the "dummy model" code?

Code in the following places is part of the "dummy model":

- `mali_kbase_model_dummy.c`
- `mali_kbase_model_dummy.h`
- Any codepath that is only enabled when `CONFIG_MALI_NO_MALI` is enabled, for example:

```
#if IS_ENABLED(CONFIG_MALI_NO_MALI)
static int kbase_api_hwcnt_set(struct kbase_context *kctx,
                             struct kbase_ioctl_hwcnt_values *values)
{
    return gpu_model_set_dummy_prfcnt_user_sample(
        u64_to_user_ptr(values->data), values->size);
}
#endif /* CONFIG_MALI_NO_MALI */
```

How do I keep the GPU powered/power it off when idle?

The GPU behaves differently between Real devices and the Virtual Device using the "dummy model" ("No Mali" configuration) :

- On a Real device, by default it uses Kbase's "demand" power policy, where it only powers on when there is work to run.
- On a Virtual Device using the "dummy model", by default it uses Kbase's "always_on" power policy, where it simulates the GPU being powered even when there is no work.

This can be significant, as some operations/syscalls provided by Kbase work differently depending on whether the GPU is powered. For example, when allocating or freeing GPU memory the GPU's MMU will be programmed when the GPU is powered on. This does not happen when the GPU is powered down.

On a Real device, the GPU can be powered on by running content. For example, a Command Buffer can be created that contains a loop, and this can then be scheduled on the GPU to keep it from going idle.

Alternatively, Kbase's power policy can be changed to power on/off the GPU (Real device), or to simulate what would happen if the GPU was powered up or powered down ("dummy model"/"No Mali" configuration)

To keep the GPU powered:

```
echo "always_on" | sudo tee /sys/class/misc/mali0/device/power_policy
```

To allow the GPU to power off shortly after becoming idle:

```
echo "coarse_demand" | sudo tee /sys/class/misc/mali0/device/power_policy
```

To confirm which power policy has been selected, you can `cat` the `power_policy` file. For example:

```
cat /sys/class/misc/mali0/device/power_policy
# For example, the output could be:
# [always_on] coarse_demand
```

The square brackets `[...]` indicate which power policy has been selected.

How do I access all IOCTLs/get around the -ENOTTY error?

After initially opening the Mali file descriptor, only a limited number of IOCTLs or other syscalls may be made, as described in `kbase_ioctl()`.

The remaining IOCTLs will give a -ENOTTY error (-ENOIOCTLCMD case in the code).

To allow other IOCTLs/certain other syscalls to be made, the following sequence must be used, in order:

- `KBASE_IOCTL_VERSION_CHECK`
 - This is used to negotiate which version of the interface is in use.
 - User-space supplies a major and minor version in the struct `kbase_ioctl_version_check`.
 - This is used to indicate which version of structures/ioctl behavior is required from userspace
 - Note that user-space may request a higher version than is available, but `kbase` will respond with its actual version.
 - An overview of the changes to the interface version are described in `mali_kbase_csf_ioctl.h`
- `KBASE_IOCTL_SET_FLAGS`
 - This is used to pass flags on context creation. See `mali_base_common_kernel.h`

Note: Older versions of Kbase required a `BASE_MEM_MAP_TRACKING_HANDLE` to be mapped after the above IOCTLs, and you may still see this in some example code. This is no longer required from driver version r43p0 onwards, however it will still accept the request to map `BASE_MEM_MAP_TRACKING_HANDLE`.

What is 'SAME_VA' / 'BASE_MEM_SAME_VA' memory?

Most of Kbase's GPU memory is allocated such that the GPU's virtual address matches the CPU's virtual address.

Such memory is known as SAME_VA memory.

Most memory will have this flag set for 64-bit userspace processes (or a similar flag, `BASE_MEM_NEED_MMIO`), even if userspace did not request it in the memory IOCTL call.

Implementing SAME_VA in Kbase involves some controls to ensure:

- The address is representable on both the CPU and GPU, checking both the GPU's `VA_BITS` register, and the CPU's `VA_BITS` macro and/or accounting for compat-processes (32-bit).
- The address is suitably aligned for its use, such as in Shader Program memory that must not cross a 4GiB boundary.
- The address does not intersect with the ranges used for 'special' handles for Kbase's mmap handler.

In addition, some memory zones are to be under Kbase control, without requiring userspace involvement to allocate the address.

This involves partitioning the available SAME_VA range to a number of zones that can be allocated from:

- A dynamic zone 'CUSTOM_VA' which can be used for GPU-driven memory allocations ('JIT', or Just-in-time GPU memory allocations).

- A number of static zones from 1<<47 upwards (64-bit userspace) or 1<<43 to 1<<44 (32-bit userspace), for `BASE_MEM_FIXED` / `BASE_MEM_FIXABLE` allocations, where userspace has some control over where in the virtual address.

The end of the 'SAME_VA' memory zone partition is determined by a call to `kbase_reg_zone_end_pfn()` on the `SAME_VA_ZONE` struct `kbase_reg_zone` returned by `kbase_ctx_reg_zone_get()`.

The above constraints to ensure SAME_VA memory works are implemented in the `get_unmapped_area` handler, `kbase_get_unmapped_area()`.

How do I allocate GPU memory on 64-bit userspace/'SAME_VA' memory?

There is a **2-step** method for allocating 'SAME_VA' memory, and this is the default for most memory when the userspace process is 64-bit.

Note: For 64-bit userspace processes, this flag does not need setting - Kbase will add the flag in to enforce it for most memory (or a similar flag, `BASE_MEM_NEED_MMAP`).

The reason for the 2 steps is because the GPU Virtual Address of 'BASE_MEM_SAME_VA' ('SAME_VA') memory is not known until the `mmap()/mmap64()` call is made.

The steps to allocate are as follows:

- Make an initial call to `KBASE_IOCTL_MEM_ALLOC` / `KBASE_IOCTL_MEM_ALLOC_EX` / `KBASE_IOCTL_MEM_ALIAS` / `KBASE_IOCTL_MEM_IMPORT`
 - A 'cookie' is returned in the `gpu_addr` IOCTL structure output parameter
- Userspace passes this 'cookie' in the `offset` parameter of `mmap()/mmap64()` to map the memory into both CPU and GPU address spaces.

There are additional zones at an address higher than the end address of the `SAME_VA` zone, which *do not* require the 2-step method for allocation. Those zones are listed in `mali_kbase_reg_track.h` (`CUSTOM_VA_ZONE` to `FIXED_VA_ZONE`).

In those cases, no `mmap()` call is required, and it is sufficient to make a single IOCTL or use Kbase-driven allocation to allocate physical pages and map them into the GPU address space. If access is required on the CPU is also required, then the GPU virtual address returned by the IOCTL should be passed as the `offset` parameter to `mmap()/mmap64()`

How do I grow memory on GPU Page Fault?

Creating GPU memory regions with the `BASE_MEM_GROW_ON_GPF` flag will allow the memory's backing to increase on handling a GPU Page Fault. The amount to grow by is controlled by the `extension` IOCTL structure parameter.

A GPU job can be used to write to such a memory region's unbacked portion, which on Real GPU platforms will cause a GPU Page Fault that can be handled by Kbase to growing the faulted memory region's physical backing.

The code related to this is:

- IRQ handling: `kbase_mmu_irq_handler()` → `kbase_mmu_interrupt()`, which enqueues on an `mali_mmu` workqueue

- `mali_mmu` workqueues use a number of worker functions. Page Faults are handled by the `kbase_mmu_page_fault_worker()` function.

Note: On a Virtual Platform, there is nothing to trigger a GPU MMU IRQ. Instead, either:

- a Real GPU platform should be used to run real GPU jobs to trigger the IRQ
- or, modifications should be made to Kbase to manually trigger the IRQ and set GPU MMU Status registers to simulate the GPU Page Fault.

Note: If KBase cannot handle the GPU Page fault, then it will terminate all GPU Queue Groups ('CSG's) on the faulting context, similar to `KBASE_IOCTL_CS_QUEUE_GROUP_TERMINATE`.

How do I change the timeout on FENCE_SIGNAL KCPU commands?

For `BASE_KCPU_COMMAND_TYPE_FENCE_SIGNAL`, the timer is defined by `KCPU_FENCE_SIGNAL_TIMEOUT_CYCLES` and represents a 10 second timeout, but capped to `MAX_TIMEOUT_MS` (4.5 seconds).

For debugging purposes:

- the timeout can be modified by writing a value in milliseconds to the debugfs file:

```
/sys/kernel/debug/mali0/fence_signal_timeout_ms
```

...up to `MAX_TIMEOUT_MS`

- fence timeouts may be completely disabled by writing `0` to:

```
/sys/kernel/debug/mali0/fence_signal_timeout_enable
```

FAQ on Mali GPUs

What is the Bifrost GPU Architecture?

Information on Arm's Bifrost GPU Architecture can be found at <https://developer.arm.com/Architectures/Bifrost>

All Bifrost GPUs are **Job Manager GPUs**:

- [Mali-G31](#), [Mali-G51](#), [Mali-G52](#), [Mali-G71](#), [Mali-G72](#), [Mali-G76](#)

What is the Valhall GPU Architecture?

Information on Arm's Valhall GPU Architecture can be found at <https://developer.arm.com/Architectures/Valhall>

The following Valhall GPUs are **Job Manager GPUs**:

- [Mali-G57](#), [Mali-G68](#), [Mali-G77](#), [Mali-G78](#), [Mali-G78AE](#).

The following Valhall GPUs are **Command Stream Frontend ('CSF') GPUs**:

- [Mali-G310](#), [Mali-G510](#), [Mali-G610](#), [Mali-G615](#), [Mali-G710](#), [Mali-G715](#), [Immortalis-G715](#).

Examples of systems using these GPUs include:

GPU	SoC	Devices
Mali-G78	Google Tensor G1	Pixel 6, Pixel 6 Pro, Pixel 6a
Mali-G710	Google Tensor G2	Pixel 7, Pixel 7 Pro, Pixel 7a, Pixel Fold, Pixel Tablet
Mali-G715	Google Tensor G3	Pixel 8, Pixel 8 Pro, Pixel 8a,
Mali-G715	Google Tensor G4	Pixel 9, Pixel 9 Pro, Pixel 9 Pro XL, Pixel 9 Pro Fold

What is the 5th Gen Arm GPU Architecture?

Information on Arm's 5th Gen GPU Architecture can be found at <https://developer.arm.com/Architectures/5th%20Gen%20Arm%20GPU%20Architecture>

All Arm 5th Gen GPUs are **Command Stream Frontend ('CSF') GPUs**.

The current list of such GPUs is as follows, but please check <https://www.arm.com> for the latest list:

- [Mali-G620](#), [Mali-G625](#), [Mali-G720](#), [Immortalis-G720](#), [Mali-G725](#), [Immortalis-G925](#).

What is a "Job Manager" GPU?

All Bifrost GPUs and some earlier Valhall GPUs are "Job Manager" GPUs.

This is where one of the GPU register interfaces is used by Kbase to run jobs on the GPU. In this case, userspace must use IOCTL syscalls to request Kbase run each GPU job.

"Job Manager" GPUs do not have any of the following:

- A Command Stream Frontend
- A GPU Firmware

Refer to [What is the Bifrost GPU Architecture?](#) and [What is the Valhall GPU Architecture?](#) for the list of "Job Manager" GPUs.

Building Kbase for "Job Manager" GPUs

In scope builds must have the following set in your Kernel's .config for use on devices with "Job Manager" GPUs:

```
CONFIG_MALI_CSF_SUPPORT=n
```

What is a "CSF" GPU?

All Arm 5th Gen GPUs, and the later Valhall GPUs are those using a Command Stream Frontend, and so are known as 'CSF' GPUs.

This is where one of the GPU register interfaces in [Job Manager GPUs](#) has been replaced by:

- A Command Stream Frontend
- A GPU Firmware that executes in the Command Stream Frontend

GPU Jobs can be initiated directly from user-space by use of Command Buffers that are shared between GPU and CPU.

Refer to [What is the Valhall GPU Architecture?](#) and [What is the 5th Gen Arm GPU Architecture?](#) for the list of "CSF" GPUs.

Building Kbase for "CSF" GPUs

In scope builds must have the following set in your Kernel's .config for use on devices with "CSF" GPUs:

```
CONFIG_MALI_CSF_SUPPORT=y
```

What is "TrustZone® Media Protection" (TZMP)?

Refer to the following for more details:

- <https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/introducing-arms-dynamic-trustzone-technology>

- <https://developer.arm.com/documentation/102104/0304/Reference-software-stacks-for-virtualization/Protected-content-with-paravirtualization>

What is "Ace-Lite" ?

Ace-Lite is a subset of the Arm AXI Bus Protocol.

Refer to the following for more details:

- <https://developer.arm.com/documentation/ka005469/1-0/?lang=en> - Arm Knowledge Base article on "What is AXI?"

What is Tile-Based rendering?

Refer to the following for more information on Tile-based rendering:

- <https://developer.arm.com/Architectures/5th%20Gen%20Arm%20GPU%20Architecture>

What is Deferred Vertex Shading (DVS)?

Deferred Vertex Shading is a feature of the 5th Gen Arm GPU Architecture. Refer to the following for more details:

- <https://community.arm.com/arm-community-blogs/b/announcements/posts/arm-gpus-built-on-new-fifth-gen-architecture>

Proprietary notice

This document is protected by copyright and other related rights and the practice or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of the document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any use, duplication or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership

relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. All rights reserved. Please follow Arm's trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.
110 Fulbourn Road, Cambridge, England CB1 9NJ.
(PRE-1121-V1.0)